

Day 7 – Building a Local RAG Pipeline for Class 10 Science Textbooks

PDF-based Knowledge Retrieval & Question-Answering System · Souvik Pal · Personal Project

Objective

The goal for today was to build a complete system, from scratch, that can read my Class 10 Science PDF textbooks and answer questions using only their content — with the answer showing exactly which chapter and page it came from. This is called RAG (Retrieval-Augmented Generation): instead of trusting an AI's memory, the system looks up the right passage first, then answers from it. Everything was built and run for free, entirely on my own laptop, with no paid tools or internet dependency once set up.

Input Data

14 Class 10 Science PDF files: Chapter 1 through Chapter 13, plus an Answers.pdf reference file. All files turned out to be digitally created (not scanned images), so no OCR (image-to-text) step was needed — the text could be pulled out directly.

Tools Used

Tool	What it does
Python 3.14.6	The programming language used for everything
venv	Keeps this project's installed packages separate from other projects
PyMuPDF (fitz)	Reads PDF files and pulls out the text, page by page
langchain-text-splitters	Cuts long text into small, overlapping pieces
sentence-transformers (all-MiniLM-L6-v2)	Converts text into number-based "meaning codes" (embeddings), runs locally
ChromaDB	A local database built for storing and searching these meaning codes
Ollama + Llama 3.2 (2B model)	A free AI model that runs on my own computer to write the final answers

How the System Works, Step by Step

Stage 1 – Reading the PDFs

Script: **read_pdf.py**. A PDF is just a file format — a computer program can't understand its content until the text is pulled out. Using PyMuPDF, I opened every PDF, counted its pages, and extracted

the text from every single page, checking a short preview each time to confirm it worked.

Stage 2 – Saving Text with Page Numbers

At first the script only printed text on screen, which is lost the moment the program closes. I fixed this by saving the text properly, and made sure every piece of text remembers exactly which chapter and page it came from. This page-tracking is what later lets the AI say "this answer came from Chapter 1, Page 3" instead of just guessing. I also added an automatic check that flags any page with almost no extractable text — a sign it might be a scanned image instead of real text.

Stage 3 – Cutting Text into Small Pieces (Chunking)

Script: **chunk_text.py**. A whole textbook page is too big and unfocused for the AI to search through well. I split every page into small chunks of about 500 characters each, with a 50-character overlap between neighbouring chunks so a sentence never gets awkwardly cut in half. I also had to fix a sorting bug: by default, the computer sorted chapter files alphabetically, so "Chapter 10" was placed right after "Chapter 1" instead of after "Chapter 9." I wrote a small fix so chapters sort in the correct numerical order.

Stage 4 – Turning Text into Searchable "Meaning Codes" (Embeddings)

Script: **build_database.py**. This is the most important idea in the whole project. Every chunk of text gets converted into a list of numbers (called an embedding) that represents its *meaning*, not just its exact words. Chunks with similar meaning end up with similar numbers, even if they use completely different words. All these number-codes, along with their chapter and page information, were saved into a local database called ChromaDB.

Stage 5 – Testing the Search, Before Adding AI

Script: **search.py**. Before connecting any AI model, I tested whether the search itself actually worked — typing a question, converting it into the same kind of number-code, and checking whether the database returned genuinely relevant chunks. This step matters because it separates two different questions: "does the search find the right information?" and "does the AI write a good answer from it?" — these can succeed or fail independently.

Stage 6 – Connecting a Free Local AI Model

Script: **ask_ai.py**. Finally, I connected Ollama, a free tool that runs AI language models directly on my computer — specifically Llama 3.2, a small 2GB model. When I ask a question, the system finds the closest-matching chunks, hands them to the AI as "context," and the AI writes a proper answer using only that context, along with the exact chapter and page it used.

Results

All 14 files were processed with zero problems — every page had clean, readable text.

What	Result
------	--------

PDFs processed	14 (13 chapters + answer key)
Total pages extracted	220
Pages with extraction problems	0
Total searchable chunks created	1,132
Cost to build and run	Free — fully offline, no paid tools

Seeing It Work End-to-End

Real test, exact output from the finished system:

Q: What is a chemical reaction?

A: A chemical reaction occurs when two or more substances combine to form a new single substance.

Sources used:

- Chapter 1, Page 1
- Chapter 1, Page 2
- Chapter 1, Page 14

The search correctly pulled all 3 sources from Chapter 1, confirming the system searches by meaning, not just matching exact words — a search for "chemical reaction" also successfully finds related text like "substances combining."

A Limitation I Noticed

The answer above is only partly complete — it describes one specific type of chemical reaction (a combination reaction), not the textbook's full general definition. Importantly, the *search* step worked perfectly; it found exactly the right pages. The small AI model simply oversimplified while writing the final sentence. This showed me that in a RAG system, "finding the right information" and "writing a good answer from it" are two separate jobs, and each needs to be checked separately — this became the direct motivation for Day 8's work.

Files Produced

- `read_pdf.py` – extracts PDF text and saves it with page numbers (Stages 1–2)
- `chunk_text.py` – splits text into small overlapping chunks, in correct chapter order (Stage 3)
- `build_database.py` – converts chunks into embeddings and stores them (Stage 4)
- `search.py` – tests search quality on its own, before adding AI (Stage 5)
- `ask_ai.py` – full pipeline: question → search → AI-written answer with sources (Stage 6)
- `extracted_json/`, `chunked_json/`, `science_vector_db/` – the data produced by each stage

Next Steps (carried into Day 8)

- Stop testing across all chapters at once — focus on one chapter and properly measure accuracy
- Build a real test set of questions with known-correct answers, instead of judging from a single example
- Understand and be able to explain how embeddings/vectors actually work, not just use them
- Look into which embedding models and datasets are considered "best" in published research